

words in a phrase. Transformers, unlike RNNs, do not always process data in the same order. The attention mechanism, on the other hand, provides context for any point in the input sequence. For example, if the input data is a natural language sentence, the transformer does not need to process the first half of the sentence before proceeding to the final. Rather, it detects the context that gives each word in the phrase its meaning. Because this feature enables higher parallelisation than RNNs, training times are reduced.

The LSTM modules can offer meaning to the sequence while remembering (or forgetting) the parts they find significant (or unimportant) with sequence-dependent data. Sentences, for example, are sequence-dependent because the order of the words is critical for comprehension. For this type of data, LSTMs are an obvious choice. An encoder and a decoder are used in Seq2Seq models. The encoder is responsible for converting the input sequence into a higher dimensional space (n-dimensional vector). The abstract vector is passed through the decoder, which converts it into a series of outputs. The output sequence could be in a different language, symbols, a duplicate of the input, or something entirely different.

Transformer, like LSTM, is architecture for converting one sequence into another using two parts (encoder and decoder). However, it differs from the previously described/existing sequence-to-sequence models in that it does not employ recurrent networks (GRU, LSTM, etc). Transformers have replaced RNN models like LSTM as the preferred model for NLP challenges. Training on larger data sets is now possible thanks to the extra training parallelisation. As a result, pre-trained systems like BERT (bidirectional encoder representations from transformers) and GPT (generative pretrained transformer) have emerged, which were trained on big linguistic data sets like the Wikipedia Corpus.

Transformers essentially combine self-attention concepts. They do exceptionally well in machine translation jobs. How do they manage to do this? A bunch of encoders and decoders are placed on top of one another. This architecture was first proposed in a work that used six encoders and six decoders. Other numbers can also be used. Each encoder has two components — the self-attention layer and the feed forward neural network, while each decoder has three components — the self-attention layer, the decoder attention layer, and the feed forward neural network. A list of input vectors is sent to the first encoder. This is frequently an output of some kind of embedding layer when we work with words. It uses a self-attention layer and subsequently a feed-forward neural network to handle them. Following that, it passes the output to the next encoder. The information

is sent to the decoders by the final encoder, who performs a similar process.

Working with Hugging Face

Hugging Face, which began as a chat app for bored teenagers, now provides open source NLP tools and has rapidly developed language processing skills. The company's goal is to develop NLP and make it more accessible to all. Hugging Face has emerged as one of the biggest AI communities in NLP open source. It has established a comprehensive library with diverse resources in order to democratise NLP and make models available to everyone. Data sets, tokenisers and transformers are just a few of the tools available to help with NLP activities, which range from chatbots to question and answer systems.

Thousands of models are available in this library, allowing a developer to execute numerous NLP tasks. Text categorisation, information retrieval, information extraction, abstractive and extractive summarisation, name-entity recognition, natural language inference, text translation, text creation, question answering, and image captioning are just a few examples.

The rapid growth of transformers has ushered in a new era of strong natural language processing technologies. Because these models are huge and expensive to train, academics and practitioners share and use pretrained versions. Hugging Face provides open source libraries with a variety of pretrained transformers that you can use with just a few lines of code. The app used to just support PyTorch, but now it also supports TF 2.0. The library has three sub-libraries.

- Transformers library
- Data sets library
- Tokenisers library

How to use Hugging Face transformer libraries

Installation: The transformers library is tested on Python 3.6+, and PyTorch 1.1.0+ or TensorFlow 2.0+. You first need to install one of these, or both TensorFlow 2.0 and PyTorch. When TensorFlow 2.0 and/or PyTorch have been installed, transformer installation can be done using *pip*, as follows:

```
!pip install transformers
```

Pipelines: The pipelines are an incredible and simple approach to utilise models for interfaces. These pipelines are objects that abstract the majority of the library's sophisticated functionality, providing a straightforward API for a variety of tasks such as named entity recognition, masked language modelling, sentiment analysis, feature extraction, and question answering.

```
from transformers import pipeline
```